

Documentação Técnica para Arguição - Global Training Technology

Este arquivo serve como material de apoio para a apresentação do TCC. Ele reúne as decisões técnicas, fluxos, endpoints, modelo de dados e respostas para perguntas prováveis da banca.

1. Visão Geral do Projeto

O projeto **Global Training Technology** é uma aplicação web para uma escola de cursos profissionalizantes. A solução integra:

- landing page pública para divulgação dos cursos;
- chatbot com IA para atendimento ao aluno;
- painel administrativo protegido por login;
- cadastro dinâmico de cursos;
- backend em FastAPI;
- agentes de IA usando Groq;
- estrutura SQLite preparada para matrículas e pagamentos;
- integração com Mercado Pago Checkout Pro.

O objetivo técnico principal é centralizar a jornada do aluno: descoberta do curso, atendimento automatizado, matrícula e pagamento online.

2. Arquitetura Geral

Camadas

Frontend

- `frontend/landing_global_training.html`
- `frontend/admin.html`
- HTML, CSS e JavaScript puro.
- Consome APIs do backend via `fetch`.

Backend

- `backend/app.py`
- FastAPI como servidor HTTP.
- Rotas públicas, rotas administrativas e rotas de pagamento.

IA

- `backend/agents/manager_agent.py`
- `backend/agents/course_agent.py`
- `backend/agents/groq_client.py`
- Uso da Groq API para respostas do chatbot.

Dados

- Cursos em arquivos JSON dentro de `backend/courses/`.
- SQLite preparado em `backend/db.py`.
- Variáveis sensíveis em `.env`.

Serviços externos

- Groq para LLM.
- Mercado Pago Checkout Pro para pagamento.

3. Fluxo Principal do Aluno

1. O aluno abre a landing page.
2. O frontend chama `GET /api/courses`.

3. O backend lê os cursos em JSON e devolve uma lista resumida.
4. O aluno pode:
 - visualizar cursos;
 - tirar dúvidas com IA;
 - abrir o modal de matrícula;
 - informar nome, e-mail e telefone;
 - iniciar checkout no Mercado Pago.
5. O frontend chama `POST /api/payments/create-checkout`.
6. O backend valida os dados, consulta o curso, cria a preferência no Mercado Pago e retorna a URL.
7. O checkout é aberto em outra aba.

4. Fluxo Administrativo

1. O administrador acessa `frontend/admin.html`.
2. A tela exige senha administrativa.
3. O frontend chama `POST /api/admin/login`.
4. O backend valida a senha usando `ADMIN_PASSWORD`.
5. Se estiver correta, o backend gera um token temporário assinado com HMAC.
6. O frontend salva o token em `localStorage`.
7. As próximas chamadas administrativas enviam:

```
Authorization: Bearer <token>
```

8. O administrador cadastra um curso.
9. O backend salva um arquivo JSON em `backend/courses/`.
10. O `ManagerAgent` é recarregado para que o chatbot reconheça o novo curso imediatamente.

5. Backend FastAPI

Arquivo principal:

```
backend/app.py
```

Responsabilidades:

- iniciar a aplicação FastAPI;
- configurar CORS;
- inicializar o `ManagerAgent` ;
- carregar cursos na inicialização;
- expor rota de chat;
- expor rota pública de cursos;
- incluir routers de admin e pagamentos.

Endpoints públicos

Método	Endpoint	Função
GET	/	Informações básicas da API
GET	/health	Verifica status, LLM e cursos carregados
GET	/api/courses	Lista pública de cursos
POST	/api/chat	Envia mensagem para o chatbot

Exemplo de chamada do chat

```
{  
  "message": "Tem curso de Python?",  
  "session_id": "web-chat-session"  
}
```

Exemplo de resposta

```
{
  "status": "success",
  "message": "Sim, temos o curso Python Básico...",
  "session_id": "web-chat-session"
}
```

6. Rotas Administrativas

Arquivo:

```
backend/admin/routes.py
```

Endpoints

Método	Endpoint	Protegido	Função
POST	/api/admin/login	Não	Login do admin
GET	/api/admin/courses	Sim	Lista cursos
POST	/api/admin/create-course	Sim	Cria curso
GET	/api/admin/course/{course_id}	Sim	Detalha curso
PUT	/api/admin/course/{course_id}	Sim	Atualiza curso
DELETE	/api/admin/course/{course_id}	Sim	Remove curso

Modelo de curso

O backend usa `CourseInput` com os campos:

```
id
name
description
price
duration_hours
level
target_audience
objectives
topics
benefits
faq
system_prompt
```

Exemplo de curso em JSON

```
{
  "id": "python_basics",
  "name": "Python Básico",
  "description": "Curso introdutório de Python para iniciantes",
  "price": 99.90,
  "duration_hours": 20,
  "level": "Iniciante",
  "target_audience": "Pessoas sem experiência em programação",
  "objectives": [
    "Entender fundamentos de programação",
    "Aprender sintaxe básica de Python"
  ],
  "topics": [
    "Variáveis e tipos de dados",
    "Estruturas de controle",
    "Funções"
  ],
  "benefits": [
    "Certificado ao final",
```

```
"Projeto prático incluído"
],
"faq": [
  {
    "question": "Preciso ter experiência anterior?",
    "answer": "Não, o curso começa do zero."
  }
],
"system_prompt": "Você é um assistente especialista no curso de Python Básico."
}
```

7. Autenticação do Admin

A autenticação é simples, mas funcional para o contexto do projeto.

Variáveis usadas

```
ADMIN_PASSWORD=troque_essa_senha
ADMIN_TOKEN_SECRET=troque_esse_segredo_por_uma_string_grande
```

Como o token é criado

1. O backend recebe a senha.
2. Compara com `ADMIN_PASSWORD` usando `secrets.compare_digest`.
3. Cria um payload:

```
admin:<expires_at>:<nonce>
```

4. Assina o payload com HMAC-SHA256 usando `ADMIN_TOKEN_SECRET`.
5. Codifica o token em Base64 URL-safe.
6. Retorna o token ao frontend.

Tempo de validade

8 horas

Pontos fortes

- A senha não fica no frontend.
- As rotas administrativas exigem Bearer Token.
- O token tem expiração.
- A assinatura usa HMAC com comparação segura.

Limitações assumidas

- Não há usuário/senha por perfil.
- O token fica em `localStorage`.
- Em produção, o ideal seria HTTPS obrigatório, cookies `HttpOnly` ou autenticação com provedor dedicado.

8. IA e Agentes

O projeto usa uma arquitetura de agentes implementada diretamente em Python.

Componentes

ManagerAgent

Arquivo:

```
backend/agents/manager_agent.py
```

Funções:

- carregar cursos JSON;

- criar um `CourseAgent` para cada curso;
- identificar se a pergunta é geral ou sobre um curso específico;
- rotear a pergunta para o agente correto;
- manter histórico por `session_id`.

CourseAgent

Arquivo:

```
backend/agents/course_agent.py
```

Funções:

- montar a base de conhecimento do curso;
- incluir descrição, preço, carga horária, objetivos, tópicos, benefícios e FAQ;
- responder perguntas com foco no curso específico.

GroqChatClient

Arquivo:

```
backend/agents/groq_client.py
```

Funções:

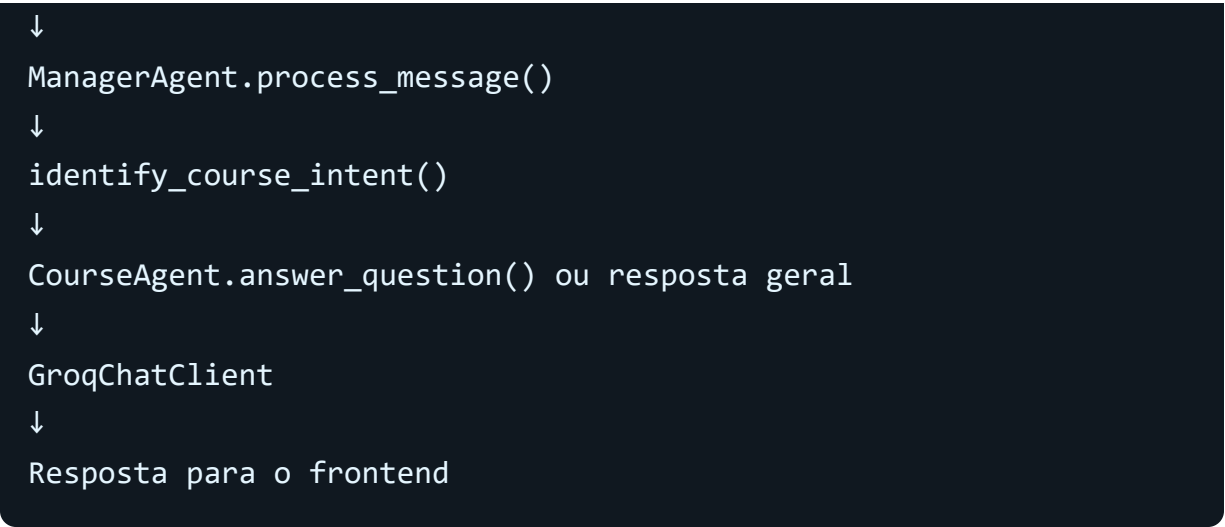
- ler `GROQ_API_KEY` ;
- configurar o modelo via `GROQ_MODEL` ;
- chamar `client.chat.completions.create` ;
- extrair o texto da resposta.

Fluxo da IA

```
Usuário pergunta
```

```
↓
```

```
POST /api/chat
```



Estratégia de roteamento

- O `ManagerAgent` tenta identificar o curso de duas formas:
- 1. busca direta por palavras-chave no nome ou ID do curso;
 - 2. se não encontrar, pede ao LLM para indicar o curso ou retornar `GENERAL`.

9. Pagamento com Mercado Pago

Arquivo:

```
backend/payments/routes.py
```

Endpoints

Método	Endpoint	Função
POST	/api/payments/create-checkout	Cria preferência de pagamento
POST	/api/payments/webhook	Recebe atualização de pagamento

Variáveis usadas

```
MERCADO_PAGO_ACCESS_TOKEN=TEST-...  
FRONTEND_BASE_URL=http://localhost:8000  
API_BASE_URL=http://localhost:8000
```

Payload enviado pelo frontend

```
{  
  "course_id": "python_basics",  
  "payer": {  
    "name": "Nome do Aluno",  
    "email": "aluno@email.com",  
    "phone": "11999999999"  
  }  
}
```

O que o backend faz

1. Valida nome e e-mail.
2. Carrega o curso pelo `course_id`.
3. Converte e valida o preço.
4. Monta os dados da preferência.
5. Envia requisição para:

```
https://api.mercadopago.com/checkout/preferences
```

6. Retorna `checkout_url`.

Segurança do pagamento

- O cartão não passa pelo sistema.

- Os dados sensíveis ficam no Mercado Pago.
- O backend só cria a preferência.
- O token do Mercado Pago fica em `.env`.

Webhook

O webhook recebe o ID do pagamento, consulta:

```
https://api.mercadopago.com/v1/payments/{payment_id}
```

Depois retorna o status recebido.

Na versão atual, o webhook está preparado para validação, mas ainda não libera matrícula automaticamente no banco.

10. Banco de Dados

Arquivo:

```
backend/db.py
```

Banco:

```
SQLite
```

Arquivo gerado:

```
backend/db.sqlite
```

Tabelas

students

```
CREATE TABLE IF NOT EXISTS students (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    email TEXT UNIQUE NOT NULL,  
    name TEXT NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

enrollments

```
CREATE TABLE IF NOT EXISTS enrollments (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    student_id INTEGER NOT NULL,  
    course_id TEXT NOT NULL,  
    status TEXT DEFAULT 'pending',  
    enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (student_id) REFERENCES students(id)  
);
```

payments

```
CREATE TABLE IF NOT EXISTS payments (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    enrollment_id INTEGER NOT NULL,  
    amount FLOAT NOT NULL,  
    status TEXT DEFAULT 'pending',  
    payment_method TEXT,  
    transaction_id TEXT,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (enrollment_id) REFERENCES enrollments(id)  
);
```

Relacionamentos

```
students 1:N enrollments  
enrollments 1:N payments
```

Decisão atual

Os cursos ficam em JSON porque:

- facilita cadastro dinâmico;
- facilita leitura pelos agentes de IA;
- simplifica a demonstração;
- reduz complexidade inicial.

O SQLite está preparado para persistir a parte transacional: alunos, matrículas e pagamentos.

11. Frontend

Landing page

Arquivo:

```
frontend/landing_global_training.html
```

Funções:

- apresenta a escola;
- carrega cursos via `GET /api/courses` ;
- renderiza cards dinamicamente;
- abre modal de matrícula;
- chama checkout;
- possui chatbot conectado ao backend.

Chamadas principais:

```
fetch(`${BACKEND_URL}/api/courses`)  
fetch(`${BACKEND_URL}/api/chat`, ...)  
fetch(`${BACKEND_URL}/api/payments/create-checkout`, ...)
```

Admin

Arquivo:

```
frontend/admin.html
```

Funções:

- login administrativo;
- armazenamento temporário do token;
- listagem de cursos;
- cadastro de cursos;
- envio de `Authorization: Bearer .`

Chave do token no navegador:

```
global_training_admin_token
```

12. Variáveis de Ambiente

Exemplo:

```
GROQ_API_KEY=your_groq_api_key_here  
GROQ_MODEL=openai/gpt-oss-120b  
ENVIRONMENT=development  
  
ADMIN_PASSWORD=troque_essa_senha  
ADMIN_TOKEN_SECRET=troque_esse_segredo_por_uma_string_grande
```

```
MERCADO_PAGO_ACCESS_TOKEN=TEST-0000000000000000  
FRONTEND_BASE_URL=http://localhost:8000  
API_BASE_URL=http://localhost:8000
```

Por que usar `.env` ?

- Evita expor chaves no código.
- Facilita trocar credenciais entre teste e produção.
- Permite publicar o projeto no GitHub sem vazarem segredos.

13. Segurança

Medidas implementadas

- `.env` ignorado pelo Git.
- `.env.example` documenta as variáveis sem expor segredos.
- Senha administrativa apenas no backend.
- Token temporário assinado com HMAC.
- Rotas administrativas protegidas.
- Checkout externo no Mercado Pago.
- Token Mercado Pago apenas no backend.
- Dados de cartão não passam pelo projeto.

Pontos de atenção para produção

- Restringir CORS para domínios específicos.
- Usar HTTPS.
- Trocar `localStorage` por cookie `HttpOnly`, se possível.
- Persistir pagamento confirmado via webhook.
- Adicionar logs estruturados.

- Adicionar rate limit no login e no chat.

14. Testes Manuais Importantes

Landing

- A página abre corretamente.
- Cursos aparecem na tela.
- Botão "Tirar dúvidas com IA" abre o chat.
- Botão "Matricular agora" abre modal.

Chat

- Pergunta geral retorna resposta.
- Pergunta sobre curso existente retorna informação do curso.
- Curso criado no admin passa a ser reconhecido.

Admin

- Senha errada bloqueia acesso.
- Senha correta libera painel.
- Sem token, rotas admin retornam erro.
- Com token, cursos são listados.
- Novo curso é salvo em JSON.

Pagamento

- Modal exige dados válidos.
- Curso inexistente retorna erro.
- Curso existente cria checkout.
- Checkout abre em nova aba.

15. Limitações Conhecidas

Estas limitações não invalidam o projeto; elas mostram consciência técnica e próximos passos.

- Matrículas e pagamentos ainda não são gravados automaticamente no SQLite pelo fluxo de checkout.
- Webhook recebe e consulta pagamento, mas ainda não atualiza matrícula.
- Não existe área do aluno.
- Admin possui senha única, sem múltiplos usuários.
- O frontend está em HTML, CSS e JS puro, sem framework.
- CORS está permissivo para facilitar desenvolvimento local.

16. Próximos Passos Técnicos

- Persistir aluno ao iniciar matrícula.
- Criar matrícula com status `pending`.
- Criar registro de pagamento com status `pending`.
- No webhook, atualizar pagamento para `approved`, `rejected` ou `pending`.
- Liberar matrícula apenas quando pagamento for aprovado.
- Criar área do aluno.
- Criar dashboard administrativo.
- Implementar edição e exclusão pela interface do admin.
- Fazer deploy com HTTPS e domínio.
- Restringir CORS em produção.

17. Perguntas Prováveis da Banca

Por que vocês usaram FastAPI?

Porque FastAPI é moderno, rápido e possui integração forte com Pydantic para validação de dados. Também gera documentação automática em `/docs`, o que ajuda no desenvolvimento e nos testes dos endpoints.

Por que os cursos estão em JSON e não no banco?

Na versão atual, os cursos em JSON simplificam o cadastro dinâmico e facilitam a leitura pelos agentes de IA. O banco SQLite foi preparado para a parte transacional, como alunos, matrículas e pagamentos. Em uma próxima etapa, os cursos também podem migrar para tabelas relacionais.

Como a IA sabe que um curso novo existe?

Quando um curso é criado no admin, o backend salva o JSON e chama `reload_manager_agent_courses`. Isso recarrega os cursos na memória do `ManagerAgent`, criando ou atualizando os agentes específicos. Assim, o chatbot passa a reconhecer o curso novo sem reiniciar o servidor.

O que é o ManagerAgent?

É o componente responsável por receber a pergunta, identificar se ela é geral ou sobre um curso específico e rotear para o agente correto.

O que é o CourseAgent?

É o agente específico de um curso. Ele monta uma base de conhecimento com descrição, preço, duração, objetivos, tópicos, benefícios e FAQ. A resposta é gerada com base nessas informações.

O sistema usa LangChain?

O projeto usa o conceito de agentes, mas a implementação atual foi feita diretamente em Python com classes próprias. A chamada ao modelo é feita pela Groq API. LangChain ou LangGraph poderiam entrar em uma evolução futura para fluxos mais complexos, ferramentas, memória persistente ou RAG.

Como funciona a segurança do admin?

O admin envia uma senha para o backend. O backend compara com a senha do `.env`. Se estiver correta, gera um token temporário assinado com HMAC. As rotas administrativas exigem esse token no header `Authorization: Bearer`.

Por que o token fica no localStorage?

Foi uma escolha simples para o contexto do projeto e para manter a aplicação em HTML/JS puro. Para produção, seria melhor avaliar cookie `HttpOnly`, HTTPS obrigatório e controle mais robusto de sessão.

O pagamento é seguro?

Sim, dentro da proposta usada. O sistema não coleta nem processa cartão. Ele cria uma preferência de pagamento no backend e redireciona o aluno para o checkout hospedado pelo Mercado Pago.

O que o webhook faz?

Ele recebe uma notificação do Mercado Pago, extrai o ID do pagamento, consulta a API do Mercado Pago e retorna o status. A próxima etapa é persistir esse status no SQLite e liberar matrícula apenas quando o pagamento for aprovado.

Por que usar SQLite?

SQLite é simples, leve e suficiente para demonstrar o modelo físico e a persistência inicial. Para produção ou maior volume, poderia ser substituído por PostgreSQL ou MySQL sem mudar a arquitetura geral.

Como vocês evitam vazar chaves no GitHub?

O `.env` está no `.gitignore`. O projeto mantém apenas `.env.example`, que mostra quais variáveis precisam existir, mas sem valores reais.

O que acontece se a Groq estiver fora do ar?

O endpoint de chat trata exceções e retorna mensagem de erro. Como melhoria, o sistema poderia ter fallback com respostas estáticas ou fila de atendimento humano.

Qual foi a parte mais complexa tecnicamente?

A integração entre cadastro dinâmico de cursos, recarregamento dos agentes e uso imediato desses dados no chatbot e na landing. Essa integração mostra que não é apenas uma página estática, mas um sistema com backend e estado dinâmico.

O que diferencia este projeto de uma landing page comum?

A landing é apenas uma das interfaces. O sistema possui backend, painel admin protegido, IA com agentes, dados dinâmicos, modelo de banco preparado e integração de pagamento.

Como vocês dividiriam a evolução do sistema?

1. Persistência completa de matrícula e pagamento.
2. Webhook finalizando matrícula aprovada.
3. Área do aluno.
4. Dashboard administrativo.
5. Deploy em produção.
6. Certificados e analytics.

18. Resumo Técnico em Uma Frase

O projeto é uma plataforma web com frontend em HTML/CSS/JS, backend em FastAPI, agentes de IA com Groq, cursos dinâmicos em JSON, banco SQLite preparado para matrículas/pagamentos e checkout seguro com Mercado Pago.